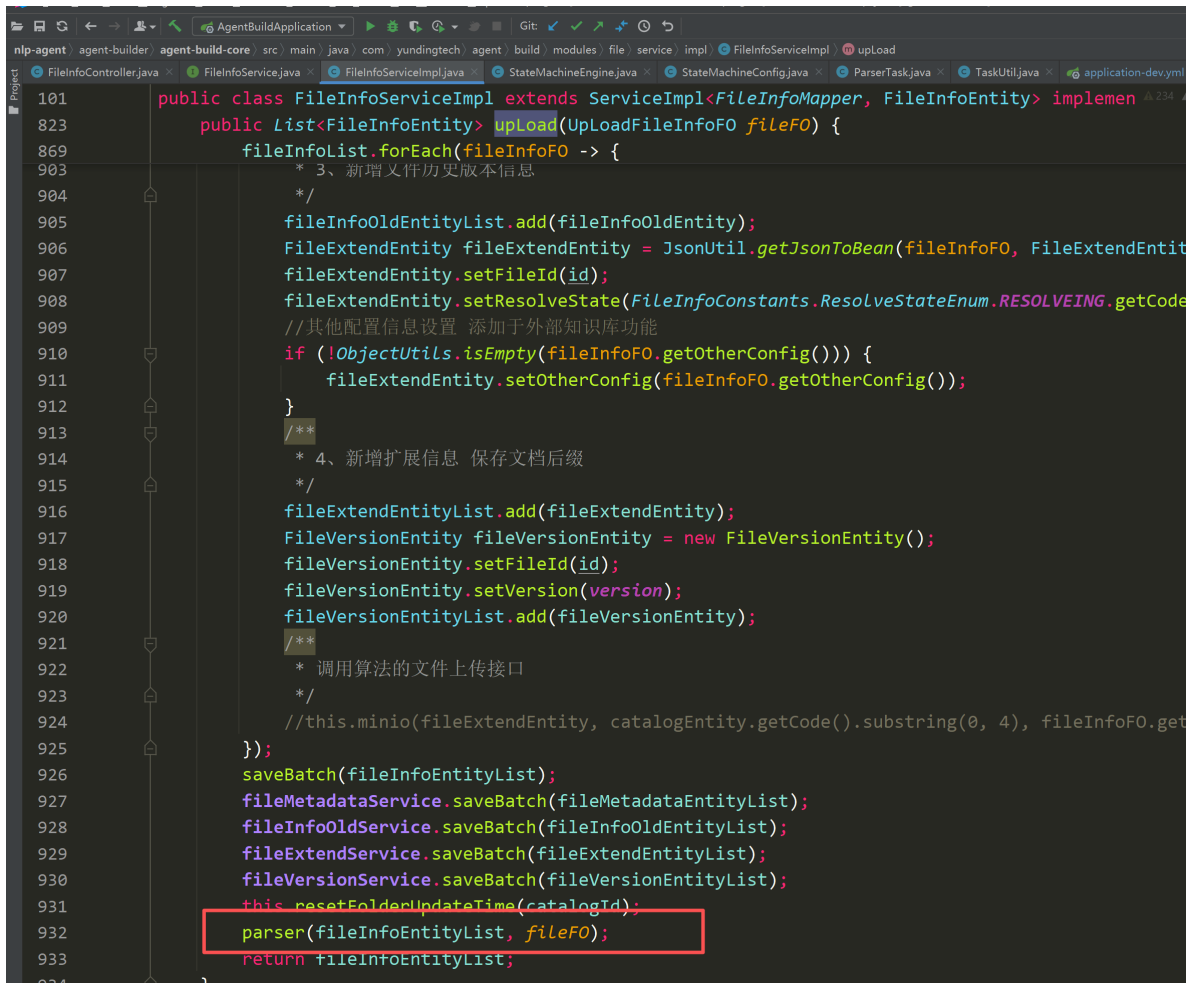


知识库文件解析

文件上传入口方法

FileInfoController#upload

FileInfoServiceImpl#upload



```
101 public class FileInfoServiceImpl extends ServiceImpl<FileInfoMapper, FileInfoEntity> implemen
823 public List<FileInfoEntity> upload(UploadFileInfoFO fileFO) {
869     fileInfoList.forEach(fileInfoFO -> {
903         * 3、新增文件历史版本信息
904         */
905         fileInfoOldEntityList.add(fileInfoOldEntity);
906         FileExtendEntity fileExtendEntity = JsonUtil.getBean(fileInfoFO, FileExtendEntit
907         fileExtendEntity.setFileId(id);
908         fileExtendEntity.setResolveState(FileInfoConstants.ResolveStateEnum.RESOLVEING.getCode
909         //其他配置信息设置 添加于外部知识库功能
910         if (!ObjectUtils.isEmpty(fileInfoFO.getOtherConfig())) {
911             fileExtendEntity.setOtherConfig(fileInfoFO.getOtherConfig());
912         }
913         /**
914         * 4、新增扩展信息 保存文档后缀
915         */
916         fileExtendEntityList.add(fileExtendEntity);
917         FileVersionEntity fileVersionEntity = new FileVersionEntity();
918         fileVersionEntity.setFileId(id);
919         fileVersionEntity.setVersion(version);
920         fileVersionEntityList.add(fileVersionEntity);
921         /**
922         * 调用算法的文件上传接口
923         */
924         //this.minio(fileExtendEntity, catalogEntity.getCode().substring(0, 4), fileInfoFO.get
925     });
926     saveBatch(fileInfoEntityList);
927     fileMetadataService.saveBatch(fileMetadataEntityList);
928     fileInfoOldService.saveBatch(fileInfoOldEntityList);
929     fileExtendService.saveBatch(fileExtendEntityList);
930     fileVersionService.saveBatch(fileVersionEntityList);
931     this.resetFolderUpdateTime(catalogId);
932     parser(fileInfoEntityList, fileFO);
933     return fileInfoList;
934 }
```

```
nlp-agent agent-builder agent-build-core src/main/java/com/yundingtech/agent/build/modules/knowledge/parser StateMachineConfig execute
FileinfoController.java FileinfoService.java FileinfoServiceImpl.java StateMachineEngine.java StateMachineConfig.java ParserTask.java TaskUtil.java application-dev.yml
47 public class StateMachineConfig {
282     public CommonResult<String> execute(ParserContext task){
283         try {
284             // 解析器信息获取
285             if (ObjectUtil.isEmpty(task.getParserId())) {
286                 Log.info("设置了通用解析器");
287                 task.setParserId(TaskUtil.DEFAULT_PARSER);
288             }
289
290             // 解析器对象
291             AgentParserEntity parserObj = agentParserService.getById(task.getParserId());
292             // 解析器支持的文件类型
293             List<AgentParserFiletypeEntity> list = agentParserFiletypeService.lambdaQuery().eq(AgentPar
294
295             if (!ObjectUtil.isEmpty(task.getFiles())) { // 上传提交
296                 for (ParserContext.FileInfo file : task.getFiles()) {
297                     Log.info("创建文件上下文信息");
298                     ParserContext submitTask = createContext(task, file);
299                     routeAndStartSingleFileTask(submitTask, parserObj, list);
300                 }
301             } else { // 重试调用时空
302                 if (task.getFileId() != null && task.getFilePath() != null) {
303                     routeAndStartSingleFileTask(task, parserObj, list);
304                 }
305             }
306             return CommonResult.success("成功");
307         } catch (Exception e) {
308             Log.info("添加解析任务异常--{}", e instanceof BusinessException ? ((BusinessException)e).getArg
309             return CommonResult.fail(msg: "异常");
310         }
311     }
312 }
```

在 routeAndStartSingleFileTask 方法中，如果需要格式转换，会调用 TaskUtil.submit(new FormatTransformTask(task,this));

```
// 非结构化文件判断
List<String> nonStructFileTypeList = Arrays.stream(nonStructMinerUParser.getSupportFileType()).s
Boolean isNonStructFile = OkHttpFileUploader.isTargetTypeList(task.getFilePath(), nonStructFile
if (isNonStructFile) { // 非结构化文件
    task.setParserCode("DefaultNonStructMinerUParser");
}
} else { // 使用配置解析器
    task.setParserCode("SelfConfigParser");
}
Log.info("调用了{}解析器", task.getParserCode());

IntegrationConfig.PdfConverter pdfConverter = integrationConfig.getPdfConverter();
List<String> nonStructFormatList = new ArrayList<>(Arrays.asList(pdfConverter.getNeedTransformType(
Boolean isNeedFormatTransform = OkHttpFileUploader.isTargetTypeList(task.getFilePath(), nonStructFo

// 文件格式处理
if (isNeedFormatTransform) { // 文件格式处理（执行结束会继续进行解析分块上传）
    TaskUtil.submit(new FormatTransformTask(task, stateMachineConfig: this));
} else { // 直接执行解析分块上传
    if (OkHttpFileUploader.isTargetTypeList(task.getFilePath(), Arrays.asList("pdf", "png", "jpg", "jp
        task.setPdfPath(task.getFilePath());
    }
    handleParsePriorityAndExecute(task);
}
```

在 handleParsePriorityAndExecute 方法中，最终也会调用 TaskUtil.submit(new ParserTask(result,this, result.getFileSize()));

```

public void handleParsePriorityAndExecute(ParserContext result) {
    //大文件逻辑 end
    Long maxLevel = result.getFileSize();
    Log.info("文件大小: {}MB", maxLevel/1024/1024);
    maxLevel = ObjectUtils.isEmpty(result.getFileSize())?TaskUtil.getMaxParser() +1: maxLevel;
    long level = (result.isTemp() ? (TaskUtil.getMinParser() - 1) : maxLevel);
    Log.info("添加解析任务---{}", 优先级--{}, 解析文件名: {}", result.getFileId(), level, result.getFileSize());
    TaskUtil.submit(new ParserTask(result, stateMachineConfig: this, result.getFileSize()));
}

```

具体流程是：

1. FileInfoServiceImpl.parser()方法中调用 stateMachineEngine.execute(parserContext)
2. StateMachineEngine.execute()转发到 StateMachineConfig.execute()
3. StateMachineConfig.execute()方法中根据文件类型和大小创建解析任务，提交给线程池执行
4. 解析任务(ParserTask、GetResultTask、FormatTransformTask)完成后，会调用 StateMachineConfig 的 success 或 fail方法来更新数据库状态

在 StateMachineConfig.execute() 方法中

1. 根据解析器类型判断使用哪种解析器
2. 检查是否需要格式转换
3. 提交解析任务到线程池执行

在提交到线程池之后：

- 1、任务执行：线程池中的线程会从队列中取出 ParserTask 任务并执行其 run() 方法。
- 2、解析过程：ParserTask.run() 方法会调用 stateMachineConfig.parser(context) 执行实际的解析逻辑。
- 3、状态判断：解析完成后，根据解析结果判断是否成功，如果成功且是大文件，则会创建 GetResultTask 并再次提交到线程池。

```

16 public class ParserTask implements Runnable, Comparable<ParserTask> {
26     @Override
27     public void run() {
28         Log.info("开始解析任务: {}, 第{}次重试, 优先级: {}", JsonUtil.getObjectToString(context), context.get
29             Thread.currentThread() = Thread.currentThread();
30             String originalName = currentThread.getName();
31             String newName = "parser-" + context.getFileId()+"-"+context.getFileName();
32             currentThread.setName(newName);
33             try {
34                 ParserContext result = stateMachineConfig.parser(context);
35                 //解析失败 执行结束, 未获得结果
36                 if (result==null || !result.isSuccess()){
37                     reStartParser(result);
38                 } else {
39                     //如果大文件 且 正常解析汇总 则等待回调 否则重试
40                     if (context.isLargeFile()) {
41                         if (context.getCurrentState().equals(TaskUtil.TaskState.PARSING) && !"Parsering".equals
42                             reStartParser(result);
43                         } else {
44                             TaskUtil.submit(new GetResultTask(result, stateMachineConfig));
45                         }
46                     } else {
47                         Log.info("文件解析任务成功, 文件Id: {}, 文件名: {}", context.getFileId(), context.getFileName(),
48                             stateMachineConfig.success(result);
49                     }
50                 }
51             } catch (Exception e) {
52                 Log.error("解析任务异常: {}--{}", context.getFileId(), e.getLocalizedMessage());
53                 reStartParser(context);
54             }
55         }
56     }

```

4、最终状态更新：当所有解析步骤都完成后，会调用 `stateMachineConfig.success(result)` 或 `stateMachineConfig.fail(context)` 方法，这两个方法会更新数据库中的文件解析状态。

整个过程是异步的，所以从上传完成到状态更新之间会有延迟，具体延迟时间取决于线程池中的任务排队情况和解

析任务的实际执行时间。